UNIT I – SOFTWARE PROCESS AND AGILE DEVELOPMENT

---

1. What is Software Engineering?

Software Engineering is the disciplined approach to developing software using systematic and planned methods. It encompasses design, implementation, testing, and long-term maintenance.

---

2. What is a Software Process?

A software process is a structured set of activities required to develop a software product. It ensures the project is executed in a controlled and orderly manner.

---

3. Why is Software Process Important?

Standardized processes help minimize defects, optimize development time, and enhance final product quality. They provide developers with a clear roadmap for the project.

---

4. What are the main activities of the software process?

1. Requirement gathering
2. Design
3. Coding
4. Testing
5. Maintenance

---

5. What are Process Models?

Process models are abstract representations or frameworks used to guide the software development lifecycle step-by-step.

Example: Waterfall model and Spiral model.

---

6. What is a Waterfall Model?

The Waterfall model is a linear-sequential life cycle model where each development phase must be completed before the next phase begins.

---

7. What is an Incremental Model?

The Incremental model involves developing software in small sections or increments, with each successive version adding new functionality.

Example: Initial version includes login features, while the next version adds payment processing.

---

8. What is a Spiral Model?

The Spiral model is an iterative model that integrates elements of design and prototyping with comprehensive risk analysis, suitable for complex projects.

---

9. What is a Prototype Model?

The Prototype model involves building a simplified sample or mockup of the system to clarify requirements before full-scale development begins.

Example: Creating a non-functional UI demo for a mobile application.

---

10. What are Specialized Process Models?

Specialized models are sophisticated frameworks tailored for specific technical requirements or unique development scenarios.

Examples: Component-based development and formal methods.

---

11. What is Component-Based Development?

This approach focuses on building software by integrating pre-existing, reusable components or modules into a single system.

Example: Integrating a third-party authentication module into a new application.

---

12. What is Agility?

Agility is the capacity of a development team to respond rapidly to changing requirements and deliver functional software in short timeframes.

---

13. What is an Agile Process?

Agile is an iterative development methodology that emphasizes continuous collaboration, small work increments, and frequent customer feedback.

---

14. What are the advantages of Agile?

1. Rapid product delivery
2. Flexibility to changes
3. Increased customer satisfaction
4. Enhanced team collaboration

---

15. What is Iteration in Agile?

An iteration is a fixed, short development cycle where a specific subset of features is developed, tested, and reviewed.

Example: A 2-week sprint cycle.

---

16. What is Scrum?

Scrum is a popular Agile framework that organizes work into time-boxed sprints and utilizes daily stand-up meetings to track progress.

---

17. What is XP?

Extreme Programming (XP) is an Agile methodology that prioritizes technical excellence, code quality, and frequent communication with stakeholders.

---

18. Name any four XP practices.

1. Pair Programming
2. Continuous Testing
3. Small Releases
4. Refactoring

---

19. What is Pair Programming?

Pair programming involves two developers working at a single workstation; one acts as the 'driver' writing code while the other acts as the 'navigator' reviewing it.

---

20. What is Refactoring?

Refactoring is the process of restructuring existing code to improve its internal design and readability without altering its external behavior.

---

21. What is Continuous Integration?

Continuous Integration is the practice of frequently merging developer code changes into a shared repository, where they are automatically built and tested.

UNIT II – REQUIREMENTS ANALYSIS AND SPECIFICATION

---

1. What is Requirement Analysis?

Requirement analysis is the process of defining and documenting user expectations for a software product before development commences.

---

2. Why is Requirement Analysis important?

Thorough analysis ensures the final system aligns with user needs and prevents costly errors during later stages of development.

---

3. What are Functional Requirements?

Functional requirements define specific services or functions the software must perform to meet user objectives.

---

4. What are Non-Functional Requirements?

Non-functional requirements specify system attributes and constraints, such as performance speed, security protocols, and overall reliability.

---

5. What is Requirement Gathering?

Requirement gathering is the systematic collection of project goals and needs through interaction with clients and end-users.

---

6. What is Formal System Specification?

Formal specification is the use of mathematical notations to precisely describe the intended behavior and constraints of a software system.

7. What is SRS and its uses?

SRS (Software Requirement Specification) is a comprehensive document that details exactly what the software system will do and how it will operate.

1. Ensures clear stakeholder understanding
2. Provides a formal client agreement
3. Serves as a developer reference
4. Facilitates the testing process

---

8. What is a FInite State Machine(FSM)?

A Finite State Machine (FSM) is a mathematical model used to design systems that exist in one of a finite number of states and transition between them.

Example: An ATM system transitioning from Idle to Card Inserted to Processing.

---

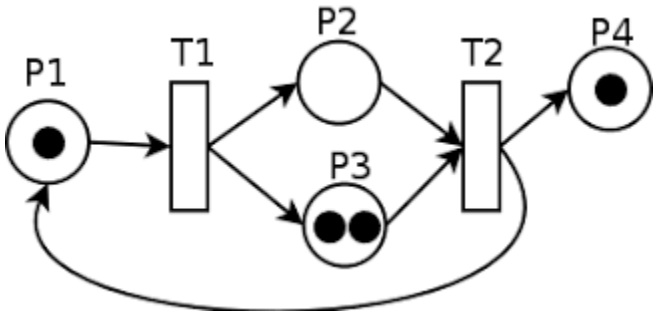
9. What are the components of FSM?

1. States
2. Inputs
3. Transitions
4. Output

---

10. What is Petri Net?

A Petri Net is a graphical modeling language used to describe and analyze concurrent, asynchronous, and parallel processes within a system.



11. Where is Petri Net used?

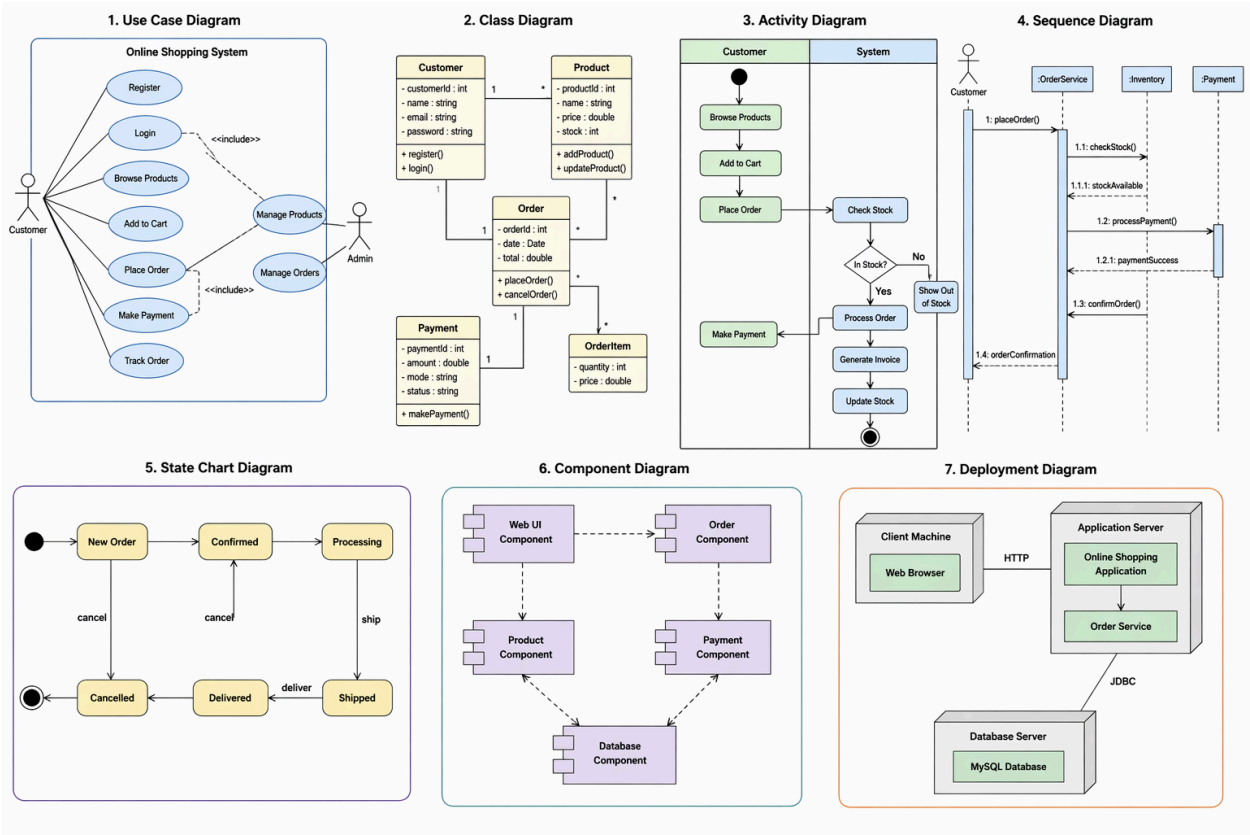
Petri Nets are frequently applied in traffic management, network protocols, and printer queue modeling.

---

12. What is UML?

UML (Unified Modeling Language) is a standardized modeling language used to visualize, specify, and document the architecture of software systems.

Sample Diagrams for UML



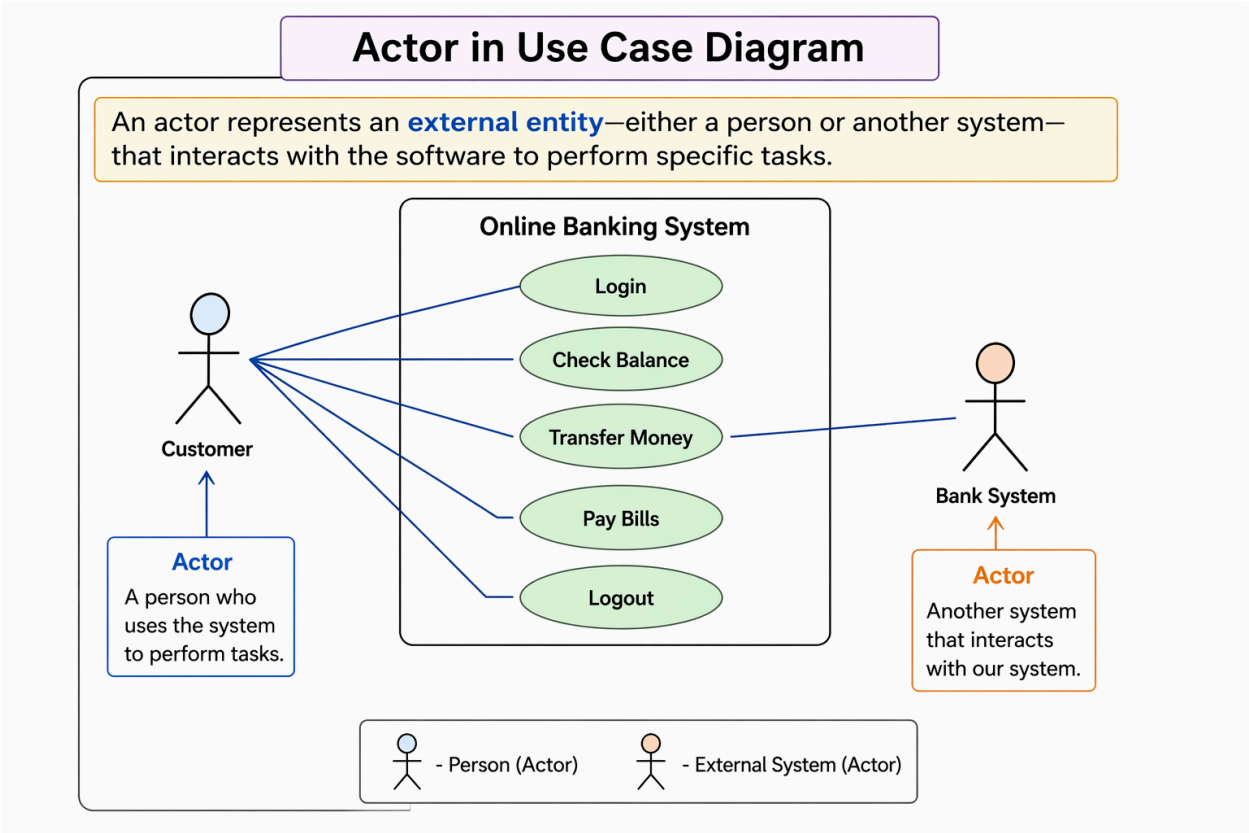
---

13. What is a Use Case Diagram?

A Use Case diagram illustrates the interactions between external actors and the system to depict the system's functional requirements.

Example: A student actor logging in to view their academic marks.

14. What is an Actor in a Use Case Diagram?



---

15. What is a Class Diagram?

A Class diagram is a static structural diagram that describes the system classes, their attributes, operations, and the relationships among objects.

---

16. What is Object in OOP?

In Object-Oriented Programming, an object is a runtime instance of a class that encapsulates both data and behavior.

Example: "Student1" is a specific object instantiated from the "Student" class.

---

17. What is an Interaction Diagram?

Interaction diagrams visualize the flow of communication and exchange of messages between different objects within a system.

---

18. What are types of Interaction Diagrams?

1. Sequence Diagram
2. Collaboration Diagram

---

19. What is a Sequence Diagram?

A Sequence diagram highlights the chronological order of messages exchanged between objects to complete a particular scenario.

---

20. What is an Activity Diagram?

An Activity diagram models the procedural workflow of a system, showing the step-by-step sequence of activities and control flows.

---

21. What is a State Chart Diagram?

A State Chart diagram describes the dynamic behavior of an object by illustrating its various states and the events that trigger transitions.

---

22. What is Functional Modeling?

Functional modeling focuses on the data transformations and functional processes that occur within a software system.

---

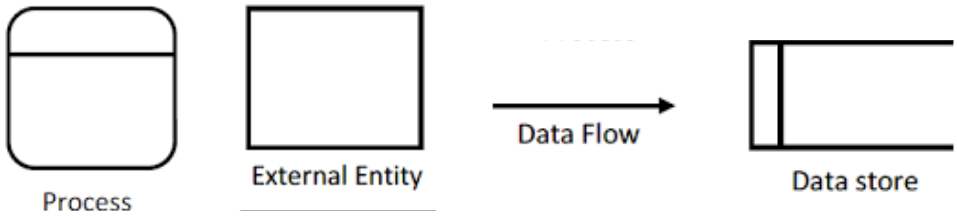
23. What is DFD?

A Data Flow Diagram (DFD) is a graphical representation that tracks how data moves through a system, from inputs to processes and storage.

---

24. What are the symbols in DFD?

1. Process
2. Data Flow
3. Data Store
4. External Entity



---

25. What is Context Level DFD?

The Context Level DFD (Level-0) provides a high-level view of the entire system as a single process, highlighting external interfaces.

---

26. What are CASE Tools?

CASE (Computer-Aided Software Engineering) tools are software applications designed to automate and support various phases of the development lifecycle.

---

27. Give examples of CASE tools.

1. Rational Rose
2. StarUML
3. Visual Paradigm

UNIT III – SOFTWARE DESIGN

---

1. What is software design?

Software design is the process of converting user requirements into a structured solution. It defines system architecture, components, and their interactions. It acts as a blueprint for coding.

---

2. What is coupling?

Coupling refers to the degree of dependency between modules. If modules depend heavily on each other, it is high coupling. Low coupling is preferred for better flexibility and maintenance.

---

3. What is cohesion?

Cohesion is the degree to which elements within a module are related. High cohesion means all functions in a module are focused on a single task. It improves clarity and reusability.

---

4. Differentiate coupling and cohesion.

Coupling	Cohesion
Coupling means the degree of dependency between two or more modules.	Cohesion means how closely related the functions inside one module are.
It measures relationships between modules.	It measures relationships within a module.
Low coupling is preferred.	High cohesion is preferred.
High coupling makes maintenance difficult.	High cohesion improves clarity and reuse.

5. What is functional independence?

Functional independence means each module performs a single function with minimal interaction with others. It is achieved by high cohesion and low coupling, improving maintainability.

---

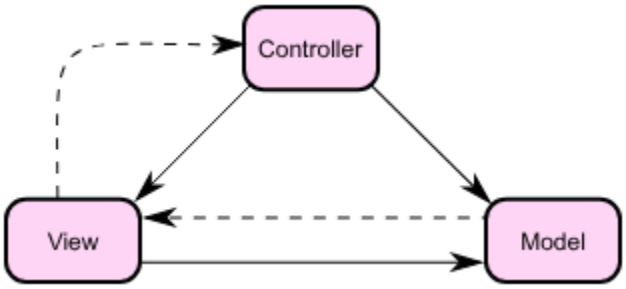
6. What is a design pattern?

A design pattern is a general reusable solution to common software design problems. It provides a standard approach to solving recurring issues in system design.

---

7. What is the MVC pattern?

MVC (Model-View-Controller) separates applications into three parts: Model (data), View (UI), and Controller (logic). It improves modularity and ease of development.



---

8. What is the Observer pattern?

Observer pattern is used when one object changes state and notifies other dependent objects. It supports one-to-many relationships between objects.

---

9. What is the Adapter pattern?

Adapter pattern allows incompatible interfaces to work together. It acts as a bridge between two different systems.

---

10. What is the Strategy pattern?

Strategy pattern allows selecting different algorithms at runtime. It makes the system flexible by changing behavior without modifying code.

---

11. What is a Proxy pattern?

Proxy pattern provides a substitute or placeholder for another object. It controls access and adds security or performance improvements.

---

12. What is the Facade pattern?

Facade pattern provides a simple interface to a complex system. It hides internal complexity and makes usage easier.

---

13. What is layered architecture?

Layered architecture divides the system into layers like presentation, business, and data. Each layer has a specific role and interacts with adjacent layers.

---

14. What is client-server architecture?

In client-server architecture, clients send requests and servers provide services. It is widely used in web applications.

---

15. What is pipe and filter architecture?

In this architecture, data flows through a series of filters. Each filter processes data and passes it to the next stage.

UNIT IV – SOFTWARE TESTING AND MAINTENANCE

---

1. What is software testing?

Software testing is the process of evaluating software to find errors or bugs. It ensures the system meets user requirements and works correctly.

---

2. What is unit testing?

Unit testing tests individual components or modules of software. It is usually performed by developers to ensure each unit works properly.

---

3. What is black box testing?

Black box testing tests software without knowing internal code. It focuses on inputs and expected outputs.

---

4. What is white box testing?

White box testing tests internal structure and logic of code. It requires knowledge of programming.

---

5. What is integration testing?

Integration testing checks how different modules work together. It ensures proper interaction between components.

---

6. What is system testing?

System testing tests the complete system as a whole. It verifies whether the system meets requirements.

---

7. What is regression testing?

Regression testing ensures that new changes do not affect existing functionality. It is done after updates or bug fixes.

---

8. What is debugging?

Debugging is the process of identifying and fixing errors in software. It is done after detecting bugs during testing.

---

9. What is program analysis?

Program analysis studies the behavior of a program to detect errors. It can be static (without execution) or dynamic (during execution).

---

10. What is symbolic execution?

Symbolic execution tests programs using symbolic inputs instead of real values. It helps in analyzing multiple execution paths.

---

11. What is model checking?

Model checking is a technique to verify system correctness using mathematical models. It ensures system behavior is correct.

---

12. What is a test case?

A test case is a set of inputs, conditions, and expected results used to test software.

---

13. What is a test suite?

A test suite is a collection of multiple test cases used to test different parts of a system.

14. What is error and defect?

An error is a human mistake in coding. A defect (bug) is the result of that error in the software.

---

15. What is software maintenance?

Software maintenance involves modifying software after delivery. It includes fixing bugs, improving performance, and updating features.

---

16. Types of software testing

- Unit testing
- Integration testing
- System testing
- Regression testing
- Acceptance testing

---

17. Advantages of regression testing

- Ensures old features still work
- Detects new bugs early
- Improves software reliability
- Supports continuous development

---

18. Uses of debugging

- Identify errors in code
- Fix bugs after testing
- Improve program performance
- Ensure correct functionality

19. Difference between black box and white box testing

Black Box Testing	White Box Testing
Testing is done without knowing internal code structure.	Testing is done by checking internal code and logic.
Focuses on input and output.	Focuses on code paths, conditions, loops.
Testers need not know programming.	The tester should know programming.
Used to test functionality.	Used to test logic and internal working.
Usually done by testers.	Usually done by developers/testers.

---

20. Applications of model checking

- Safety-critical systems (aircraft, medical)
- Protocol verification
- Hardware design checking

UNIT V – PROJECT MANAGEMENT

---

1. What is software project management?

It is the process of planning, organizing, and managing software projects. It ensures completion within time, cost, and quality.

---

2. What is software configuration management?

SCM manages and controls changes in software. It helps track versions and maintain consistency.

---

3. What is project scheduling?

Project scheduling defines timelines for tasks and activities. It helps in completing work on time.

---

4. What is DevOps?

DevOps is a practice that combines development and operations. It improves collaboration and speeds up delivery.

---

5. What is the motivation for DevOps?

DevOps reduces delays between development and deployment. It improves efficiency, speed, and quality.

---

6. What are operations in DevOps?

Operations involve deployment, monitoring, and maintenance of applications. It ensures smooth functioning after release.

---

7. What is the deployment pipeline?

A deployment pipeline is an automated process for building, testing, and deploying software.

8. What is cloud as a platform?

Cloud provides infrastructure and services over the internet. It allows scalable and flexible application deployment.

Cloud Service Models are:

- IaaS – Infrastructure as a Service
- PaaS – Platform as a Service
- SaaS – Software as a Service

What is IaaS?

IaaS provides virtual machines, storage, and networking through the internet. Users manage OS and applications.

Example: Amazon Web Services virtual servers.

What is PaaS?

PaaS provides a platform to develop, test, and deploy applications without managing hardware.

Example: Google Cloud.

What is SaaS?

SaaS provides ready-to-use software through a browser or app.

Example: Google Workspace.

---

9. What is the build process?

The build process converts source code into executable software. It includes compiling and packaging.

---

10. What is testing in DevOps?

Testing in DevOps is automated and continuous. It ensures quality at every stage of development.

---

11. What is deployment?

Deployment is the process of releasing software to users. It can be manual or automated.

---

12. What are DevOps tools?

DevOps tools support automation and collaboration. Examples: Jenkins, Docker, Kubernetes.

---

13. What is Continuous Integration (CI)?

CI is the practice of merging code frequently into a shared repository. It helps detect errors early.

---

14. What is Continuous Deployment (CD)?

CD automatically releases code to production after testing. It ensures fast delivery.

---

15. Types of project scheduling techniques

- Gantt Chart
- PERT (Program Evaluation Review Technique)
- CPM (Critical Path Method)

---

16. Difference between CI and CD

CI	CD
CI stands for **Continuous Integration**	CD stands for **Continuous Delivery** or **Continuous Deployment**
Developers frequently merge code into a shared repository	Code is automatically prepared/released to testing or production
Detects bugs early through automated tests	Delivers updates quickly to users
Happens before deployment stage	Happens after CI stage

17. Advantages of DevOps

- Faster delivery
- Better collaboration
- Reduced errors
- Continuous improvement
- High-quality software

---

18. Uses of cloud in DevOps

- Application hosting
- Storage and backup

- Scalable infrastructure
- Continuous deployment support

19. Applications of Software Configuration Management

- Version control (Git)
- Team collaboration
- Change tracking
- Software maintenance
- Release management